

Advanced JAVA and J2EE – III UNIT

2 marks:

1. Write the any two differences between StringBuffer and StringBuilder class.

StringBuffer	StringBuilder
Synchronized (thread-safe).	Not synchronized (not thread-safe).
Slower performance due to synchronization overhead.	Faster performance.

2. Write the purpose of + operator in string handling with an example.

The '+' operator concatenates two strings, producing a String object as the result. This allows you to chain together a series of + operations.

Eg: String age="9";
 String s = "He is " + age + " years old.";
 System.out.println(s);
 This displays the string —He is 9 years old.‖

3. Write two ways to create a String object using the String constructors, and provide an example for each.

- To create an empty String, you call the default constructor. For example,
 String s = new String();
 will create an instance of String with no characters in it.
- To create a String initialized by an array of characters, use the constructor shown here:

String(char chars[])

Eg: char chars[] = { 'a', 'b', 'c' };
 String s = new String(chars); //This constructor initializes s with the string "abc".

4. What is the purpose of the toString() method in the context of strings.

In Java, the toString() method is used to provide a string representation of an object. To implement toString(), simply return a String object that contains the human-readable string that appropriately describes an object of your class.

5. How to initialize String objects using string literals?

In Java, string objects can be initialized using string literals. A string literal in Java is a sequence of characters enclosed in double quotes (").

Eg: String message = "Hello, world!";

6. What are the different ways to extract individual characters from a string? Give the syntax.

To extract a single character from a String: char charAt(int where)

To extract more than one character at a time:

void getChars(int sourceStart, int sourceEnd, char target[], int targetStart)

To store the characters in an array of bytes: byte[] getBytes()

To convert all the characters in a String object into a character array: char[] toCharArray()

7. Provide the syntax and an example for charAt() method to extract characters from a string.

Syntax: char charAt(int where)

Example: char ch;

 ch = "abc".charAt(1);

 assigns the value —"b" to ch.

8. Write the purpose and syntax of the `regionMatches()` method in string handling.

The `regionMatches()` method compares a specific region inside a string with another specific region in another string. There is an overloaded form that allows you to ignore case in such comparisons. Here are the general forms for these two methods:

```
boolean regionMatches(int startIndex, String str2, int str2StartIndex, int numChars)
boolean regionMatches(boolean ignoreCase, int startIndex, String str2,
                      int str2StartIndex, int numChars)
```

9. Demonstrate the usage of the `startsWith()` and `endsWith()` methods in string handling.

The `startsWith()` method determines whether a given String begins with a specified string. Conversely, `endsWith()` determines whether the String in question ends with a specified string.

For example, `"Foobar".endsWith("bar")` and `"Foobar".startsWith("Foo")` are both true.

10. What's the difference between `equals()` and `==` ?

The `equals()` method compares the characters inside a String object and returns true if all characters match in both strings, else returns false. The `==` operator compares two object references to see whether they refer to the same instance.

11. Describe the functionalities of the `indexOf` and `lastIndexOf` methods used for string manipulation.

- `indexOf()` Searches for the first occurrence of a character or substring.
Syntax: `int indexOf(int ch)`
- `lastIndexOf()` Searches for the last occurrence of a character or substring.
Syntax: `int lastIndexOf(int ch)`

In all cases, the methods return the index at which the character or substring was found, or `-1` on failure.

12. What are the two forms of the `substring()` method and what do their arguments represent?

- String substring(int startIndex).* This form returns the substring that starts at *startIndex* and runs to the end of the invoking StringBuffer object.
- String substring(int startIndex, int endIndex).* It returns the substring that starts at *startIndex* and runs through *endIndex-1*.

13. Describe the two forms of the `replace()` method and their functionalities.

- String replace(char original, char replacement)* replaces all occurrences of one character in the invoking string with another character. Here, original specifies the character to be replaced by the character specified by replacement.
- String replace(CharSequence original, CharSequence replacement)* replaces one character sequence with another.

14. What is the purpose of the `valueOf()` method in Java and how does it relate to the `toString()` method?

The `valueOf()` method converts data from its internal format into a human-readable form. Any object that you pass to `valueOf()` will return the result of a call to the object's `toString()` method. In fact, you could just call `toString()` directly and get the same result.

15. How does `StringBuffer` differ from `String` in terms of mutability and growth?

String represents fixed-length, immutable character sequences. In contrast, StringBuffer represents growable and writeable character sequences. StringBuffer may have characters and substrings inserted in the middle or appended to the end. StringBuffer will automatically grow to make room for such additions.

16. What is the purpose of the ensureCapacity() method in StringBuffer? give its general form

If you want to preallocate room for a certain number of characters after a StringBuffer has been constructed, you can use ensureCapacity() to set the size of the buffer. This is useful if you know in advance that you will be appending a large number of small strings to a StringBuffer.

ensureCapacity() has this general form: *void ensureCapacity(int capacity)*. Here, capacity specifies the size of the buffer.

17. How does the setLength() method modify the length of a StringBuffer and what happens to existing data when the length is changed?

setLength() method is used to set the length of the buffer within a StringBuffer object.

void setLength(int len)

When you increase the size of the buffer, null characters are added to the end of the existing buffer.

If you call setLength() with a value less than the current value returned by length(), then the characters stored beyond the new length will be lost.

18. What do the charAt() and setCharAt() methods do in StringBuffer?

The value of a single character can be obtained from a StringBuffer via the *charAt()* method.

char charAt(int where) , where specifies the index of the character being obtained.

You can set the value of a character within a StringBuffer using setCharAt().

void setCharAt(int where, char ch) , where specifies the index of the character being set, and ch specifies the new value of that character.

19. What does the append() method do in StringBuffer? Which function is called for each parameter to obtain its string representation.

The append() method concatenates the string representation of any other type of data to the end of the invoking StringBuffer object. Here are a few of its forms:

StringBuffer append(String str), *StringBuffer append(int num)*, *StringBuffer append(Object obj)*

String.valueOf() is called for each parameter to obtain its string representation. The result is appended to the current StringBuffer object.

20. What does the insert() method do in StringBuffer and how is it different from append()?

The insert() method inserts a specified string at a given *index* position within the existing string buffer content.

The append() method appends a given string to the *end* of the existing string buffer content.

21. How does String Builder differ from StringBuffer?

Refer Q.No. 1

22. What are the roles of the Stub and Skeleton objects in RMI?

The communication between client and server is handled by using two intermediate objects: Stub object (on client side) and Skeleton object (on server-side).

23. What is RMI?

The RMI (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. The RMI allows an object to invoke methods on an object running in another JVM. The RMI provides remote communication between the applications using two objects stub and skeleton.

24. What is Syntactic transparency?

Syntactic transparency implies that there should be a similarity between the remote process and a local procedure.

25. What is Semantic transparency?

Semantic transparency implies that there should be similarity in the semantics i.e. meaning of a remote process and a local procedure.

26. What are the requirements for a class to be serializable?

- Implement the Serializable interface.
- Serializable objects can be written to an output stream (e.g., a file or network socket) using an ObjectOutputStream, and later read from an input stream using an ObjectInputStream.

27. State two advantages of using distributed computing over centralized computing.

- **Scalability:** Distributed systems are generally more scalable than centralized systems, as they can easily add new devices or systems to the network to increase processing and storage capacity.
- **Reliability:** Distributed systems are often more reliable than centralized systems, as they can continue to operate even if one device or system fails.

28. State any four key characteristics that define a distributed computing system.

- **Multiple Devices or Systems:** Processing and data storage is distributed across multiple devices or systems.
- **Peer-to-Peer Architecture:** Devices or systems can act as both clients and servers, as they can both request and provide services to other devices or systems in the network.
- **Shared Resources:** Resources such as computing power, storage, and networking are shared among the devices or systems in the network.
- **Horizontal Scaling:** Scaling can be done through hardware upgrades or by adding additional devices or systems to the network.

29. List the elements used in the working of RPC.

- Client • Client Stub • RPC Runtime • Server Stub • Server

30. Why is the RMI Registry important in Remote Method Invocation (RMI) applications?

The RMI registry is a simple naming service provided by Java RMI that allows clients to look up remote objects by name. It acts as a central registry where remote objects are bound to names, making them accessible to clients. The server creates an RMI registry, and clients use it to locate remote objects.

31. Write any two limitations of distributed computing.

- **Complexity:** Distributed systems can be more complex as they involve multiple devices or systems that need to be coordinated and managed.
- **Security:** It can be more challenging to secure a distributed system, as security measures must be implemented on each device or system to ensure the security of the entire system.

32. When does RMI callback occur?

An RMI callback occurs when the client of one service passes an object, that is the proxy, for another service. The recipient can then call methods in the object it received, and be calling back (hence the name) to where it came from.

Long Answer Questions

1. With syntax and example, explain the following String class methods

- a. split() b. regionMatches()**

a. split(): This method splits the given string into an array of substrings based on the specified delimiter. The delimiter is a substring that separates the original string into pieces.

Syntax: `String[] split(String regex)`

`String[] split(String regex, int max)`

where each part is delimited by the regular expression passed in *regex*. The number of pieces is specified by *max*.

Example: String str = "apple,banana,orange";
 String[] fruits = str.split(",");
 for (String fruit : fruits) {
 System.out.println(fruit); } }

b. regionMatches() : The `regionMatches()` method compares a specific region inside a string with another specific region in another string. There is an overloaded form that allows you to ignore case in such comparisons. Here are the general forms for these two methods:

```
boolean regionMatches(int startIndex, String str2, int str2StartIndex, int numChars)
boolean regionMatches(boolean ignoreCase, int startIndex, String str2,
                      int str2StartIndex, int numChars)
```

For both versions, *startIndex* specifies the index at which the region begins within the invoking String object. The String being compared is specified by *str2*. The index at which the comparison will start within *str2* is specified by *str2StartIndex*. The length of the substring being compared is passed in *numChars*. In the second version, if *ignoreCase* is true, the case of the characters is ignored. Otherwise, case is significant.

Example: String str1 = "Hello World";
 String str2 = "World";
 boolean result = str1.regionMatches(6, str2, 0, 5);
 System.out.println("Region matches: " + result);

2. With syntax and example, explain the following String class methods

a. getChars() b. replace()

a. getChars(): This method is used to extract more than one character at a time from a string and copies them into the specified character array. It has this general form:

```
void getChars(int sourceStart, int sourceEnd, char target[ ], int targetStart)
```

Here, *sourceStart* specifies the index of the beginning of the substring, and *sourceEnd* specifies an index that is one past the end of the desired substring. The array that will receive the characters is specified by *target*. The index within *target* at which the substring will be copied is passed in *targetStart*.

Example: String str = "Hello, World!";
 char[] charArray = new char[5];
 str.getChars(0, 5, charArray, 0);
 System.out.println("Copied characters: " + Arrays.toString(charArray));

b. replace() : This method replaces occurrences of a specified character or character sequence in the given string with another character or character sequence. It has two forms:

i. *String replace(char original, char replacement)* replaces all occurrences of one character in the invoking string with another character. Here, *original* specifies the character to be replaced by the character specified by *replacement*. The resulting string is returned.

ii. *String replace(CharSequence original, CharSequence replacement)* replaces one character sequence with another.

Example: String s = "Hello".replace('l', 'w'); //puts the string "Hewwo" into s.

3. With syntax and example, explain the following StringBuffer class methods

a. insert() b. deleteCharAt()

a. insert() : This method inserts a specified string at a given *index* position within the existing string buffer content. These are a few of its forms:

```
StringBuffer insert(int index, String str)
StringBuffer insert(int index, char ch)
StringBuffer insert(int index, Object obj)
```

Here, *index* specifies the index at which point the string will be inserted into the invoking StringBuffer object.

Example:

```
StringBuffer buffer = new StringBuffer("Hello World!");
        buffer.insert(6, ", ");
        System.out.println("Inserted string: " + buffer);
```

b. deleteCharAt(): The deleteCharAt() method deletes the character at the specified index from the StringBuffer object.

Syntax: `StringBuffer deleteCharAt(int loc)`
Here, *loc* specifies the index of the character to be deleted.

Example:

```
StringBuffer buffer = new StringBuffer("Hello World!");
        buffer.deleteCharAt(5);
        System.out.println("String after deletion: " + buffer);
```

4. With syntax and example, explain the following StringBuffer class methods

a. substring() b. lastIndexOf()

a. substring() : You can obtain a portion of a StringBuffer by calling substring(). It has the following two forms:

- String substring(int startIndex).* This form returns the substring that starts at *startIndex* and runs to the end of the invoking StringBuffer object.
- String substring(int startIndex, int endIndex).* It returns the substring that starts at *startIndex* and runs through *endIndex*–1.

Example:

```
StringBuffer str = new StringBuffer ("Hello World!");
String sub1 = str.substring(6);
String sub2 = str.substring(0, 5);
System.out.println("Substrings:" +sub1+ ", " +sub2);
//Output:World!,Hello
```

b. lastIndexOf() : Searches for the last occurrence of a character or substring. It has the following two forms:

- int lastIndexOf(String str) :* Searches the invoking StringBuffer for the last occurrence of *str*. Returns the index of the match, or –1 if no match is found.
- int lastIndexOf(String str, int startIndex) :* Searches the invoking StringBuffer for the last occurrence of *str*, beginning at *startIndex*. Returns the index of the match, or –1 if no match is found.

Example:

```
StringBuffer str = new StringBuffer ("Hello World!");
int index1 = str.lastIndexOf("o");
int index2 = str.lastIndexOf("o", 7);
System.out.println("Indexes: " +index1+ ", " +index2); //Output:7,7
```

5. Explain how to concatenate string with other data types?

You can concatenate strings with other types of data.

- For example,

```
int age = 9;
String s = "He is" +age+ "years old.";
System.out.println(s);
```

Output: `He is 9 years old.`

- In this case, *age* is an int rather than another String. The int value in *age* is automatically converted into its string representation within a String object. This string is then concatenated.
- The compiler will convert an operand to its string equivalent whenever the other operand of the + is an instance of String.

- Consider the following:

```
String s = "four: " + 2 + 2;
System.out.println(s);
```

This fragment displays *four: 22* rather than the *four: 4*. Here's why:

- Operator precedence causes the concatenation of *four* with the string equivalent of 2 to take place first. This result is then concatenated with the string equivalent of 2 a second time.
- To complete the integer addition first, you must use parentheses, like this:

```
String s = "four: " + (2 + 2); Now s contains the string "four: 4".
```

6. Explain character extraction methods of String class with an example.

i. **charAt()** : To extract a single character from a String, you can refer directly to an individual character via the `charAt()` method. It has this general form: *char charAt(int where)*

Here, *where* is the index of the character that you want to obtain.

Example:

```
char ch;
ch = "abc".charAt(1); //assigns the value "b" to ch.
```

ii. **getChars()** : Refer Q.No. 2 (a)

iii. **getBytes()** : To store the characters in an array of bytes. It has this general form

byte[] getBytes()

Example:

```
String message = "Hello, world!";
byte[] bytes1 = message.getBytes();
```

iv. **toCharArray()** : To convert all the characters in a String object into a character array. It has this general form: *char[] toCharArray()*

Example:

```
String str = "Hello";
char[] charArray = str.toCharArray();
System.out.println("Character array: " + Arrays.toString(charArray));
//Output: [H,e,l,l,o]
```

7. Explain any four string comparison methods with an example.

i. **equals()** : This method compares the content of two strings and returns 'true' if they contain the same characters in the same order, otherwise 'false'. The comparison is case-sensitive. It has this general form:

boolean equals(Object str)

Here, *str* is the String object being compared with the invoking String object.

Example:

```
String str1 = "hello";
String str2 = "HELLO";
boolean result2 = str1.equals(str2); // false
```

ii. **equalsIgnoreCase()** : This method compares the content of two strings ignoring their case and returns 'true' if they are equal, otherwise 'false'. It has this general form:

boolean equalsIgnoreCase(String str)

Here, *str* is the String object being compared with the invoking String object.

Example:

```
String str1 = "hello";
String str2 = "HELLO";
boolean result2 = str1.equals(str2); // true
```

iii. **regionMatches()** : Refer Q.No. 1(b)

iv. **compareTo()** : It compares the strings with the argument and returns an integer value based on the comparison. It returns a negative value if the calling string is less than the argument string (according to the natural ordering), zero if they are equal, and a positive value if the calling string is greater. It has this general form: *int compareTo(String str)*

Example: String str1 = "apple";
 String str2 = "banana";
 int result = str1.compareTo(str2); // Negative value (-)

8. Explain the difference between the `indexOf()` and `lastIndexOf()` methods in the `String` class. Give an example for each method to illustrate their functionality.

- **`indexOf()`** - Searches for the first occurrence of a character or substring.
 Returns the index of the first occurrence or -1 on failure.
 Searches the string from the beginning to the end.
 Syntax: `int indexOf(int ch), int indexOf(String str)`

Example: String str = "hello world";
 int index = str.indexOf('o');
 System.out.println("Index of 'o': " + index); // Output: Index of 'o': 4

- **`lastIndexOf()`** - Searches for the last occurrence of a character or substring.
 Returns the index of the last occurrence or -1 on failure.
 Searches the string from the end to the beginning.
 Syntax: `int lastIndexOf(int ch), int lastIndexOf(String str)`

Example: String str = "hello world";
 int index = str.lastIndexOf('o');
 System.out.println("Index of 'o': " + index); // Output: Index of 'o': 7

9. Describe two common approaches for modifying Strings in Java with an example.

a. **`substring()`** : Refer Q.No. 4(a)

b. **`concat()`** : The `concat()` method is used to concatenate one string with another string. It returns a new string that represents the concatenation of the two strings.

Syntax: String concat(String *str*)

Example: String str1 = "Hello";
 String str2 = " World!";
 String result = str1.concat(str2);
 System.out.println(result); // Output: Hello World!

10. Explain the use of `charAt()` and `setCharAt()` methods with suitable example.

The value of a single character can be obtained from a `StringBuffer` via the `charAt()` method. It returns the character at the specified index within the `StringBuffer` object.

char charAt(int where) , where specifies the index of the character being obtained.

You can set the value of a character within a `StringBuffer` using `setCharAt()`.

void setCharAt(int where, char ch) , where specifies the index of the character being set, and *ch* specifies the new value of that character.

Example: class `setCharAtDemo` {
 public static void main(String args[]) {
 StringBuffer sb = new StringBuffer("Hello");
 System.out.println("buffer before = " + sb);
 System.out.println("charAt(1) before = " + sb.charAt(1));
 sb.setCharAt(1, 'i');
 System.out.println("buffer after = " + sb);
 System.out.println("charAt(1) after = " + sb.charAt(1));
 }
 }

Output: buffer before = Hello
 charAt(1) before = e
 buffer after = Hillo
 charAt(1) after = i

11. Explain the purpose of the `valueOf()` method in the `String` class with an example.

The `valueOf()` method in the `String` class is used to convert different data types into string representation. It provides a convenient way to create strings from various types of data.

Purpose of `valueOf()` Method:

- **Converts Other Data Types to Strings:** The `valueOf()` method converts data types such as integers, floating-point numbers, characters, booleans, and objects into their string representation.
- **String Concatenation:** It's often used in string concatenation operations where non-string data needs to be combined with strings.

Here are a few of its forms:

```
static String valueOf(double num)
static String valueOf(long num)
static String valueOf(Object ob)
static String valueOf(char chars[ ])
```

Example: `String str = String.valueOf(10);` // Convert integer to string-

In this example, the `valueOf()` method is used to convert the integer value `10` into its string representation. The result is assigned to the string variable `str`. Now, `str` holds the string value `"10"`, which can be used in string operations.

12. What is the use of `replace()` method? Explain.

You can replace one set of characters with another set inside a **`StringBuffer`** object by calling **`replace()`**. Its signature is shown here:

```
StringBuffer replace(int startIndex, int endIndex, String str)
```

The substring being replaced is specified by the indexes `startIndex` and `endIndex`. Thus, the substring at `startIndex` through `endIndex-1` is replaced. The replacement string is passed in `str`. The resulting **`StringBuffer`** object is returned.

Example:

```
class replaceDemo {
    public static void main(String args[]) {
        StringBuffer sb = new StringBuffer("This is a test.");
        sb.replace(5, 7, "was");
        System.out.println("After replace: " + sb);
    }
}
```

Output: After replace: This was a test.

13. With an example explain the four methods of `StringBuffer` class.

Refer Q.No. 3, 10, 12,

14. What is the usage of `delete()` and `deleteCharAt()` methods? Explain them using their syntax and an example.

i. **`delete()`** : The `delete()` method deletes a sequence of characters from the invoking object.

Syntax: `StringBuffer delete(int startIndex, int endIndex)`

Here, `startIndex` specifies the index of the first character to remove, and `endIndex` specifies an index one past the last character to remove.

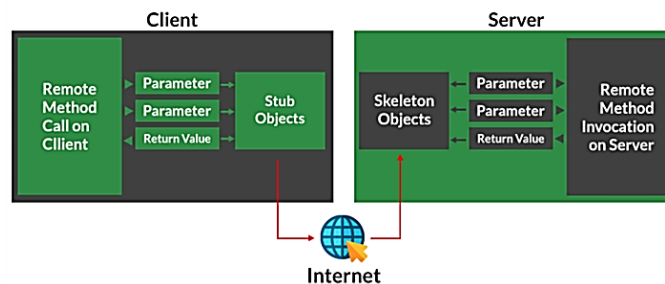
Example:

```
StringBuffer buffer = new StringBuffer("Hello World!");
buffer.delete(6, 11);
System.out.println("After deletion: " + buffer); //Output: Hello!
```

ii. **`deleteCharAt()`** : Refer Q.No. 3(b)

15. Explain in detail the method that is used by the RMI client to connect to remote RMI servers?

The communication between client and server is handled by using two intermediate objects: Stub object (on client side) and Skeleton object (on server-side) as also can be depicted from below media as follows:



1. **Define the Remote Interface:** You define a Java interface that extends the `java.rmi.Remote` interface. This interface defines the methods that the client can invoke on the remote object.
2. **Implement the Remote Object:** You implement the remote interface on the server side. This class will extend `java.rmi.server.UnicastRemoteObject` or `java.rmi.server.RemoteObject` and implement the methods defined in the remote interface. The server class provides the implementation for the methods declared in the remote interface.
3. **Create and Start the RMI Registry:** The server creates an RMI registry, which acts as a central registry for remote objects. The registry listens for incoming requests on a specific port.
4. **Bind the Remote Object to the Registry:** The server binds the remote object to the RMI registry using a unique name. This makes the remote object accessible to clients by its name.
5. **Lookup the Remote Object on the Client Side:** The client looks up the remote object in the RMI registry using the naming service (`java.rmi.Naming` or `java.rmi.registry.LocateRegistry`). The client obtains a reference to the remote object, which it can then use to invoke remote methods.
6. **Invoke Remote Methods:** The client invokes methods on the remote object reference obtained from the RMI registry. RMI handles the communication details, including parameter organizing, network communication, and error handling, transparently to the client.
7. **Handle Exceptions:** Both the client and server should handle `java.rmi.RemoteException` and any other application-specific exceptions that may occur during remote method invocation.

16. Explain the concepts of object persistence and serialization in Java. Provide a brief overview of each concept, highlighting their significance in software development.

Object Persistence:

- Object persistence refers to the ability to store and retrieve objects beyond the lifetime of the application's execution.
- With object persistence, the state of an object can be saved to a persistent storage medium (such as a database, file system, or cloud storage) and later can be restored, allowing the application to work with the same data across different runs or even different instances of the application.
- Persistence is essential for applications that need to maintain data integrity, share data between multiple users or instances, or store data for long-term use.

Serialization:

- Serialization is the process of converting an object into a stream of bytes, which can be easily stored or transmitted and later reconstructed to create an identical copy of the original object.
- In Java, serialization is achieved by implementing the `Serializable` interface, which is a marker interface indicating that the class is serializable.

- Serializable objects can be written to an output stream (e.g., a file or network socket) using an ObjectOutputStream, and later read from an input stream using an ObjectInputStream.
- Serialization allows objects to be easily stored to disk, transmitted over a network, or saved to a database, enabling object persistence.

17. Explain four key challenges associated with distributed computing systems.

- **Network latency:** The communication network in a distributed system can introduce latency, which can affect the performance of the system.
- **Distributed coordination:** Distributed systems require coordination among the nodes, which can be challenging due to the distributed nature of the system.
- **Security:** Distributed systems are more vulnerable to security threats than centralized systems due to the distributed nature of the system.
- **Data consistency:** Maintaining data consistency across multiple nodes in a distributed system can be challenging.

18. Describe the three key components that make up a Distributed Computing System.

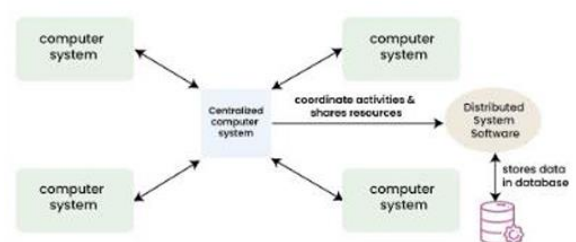
- **Devices or Systems:** The devices or systems in a distributed system have their own processing capabilities and may also store and manage their own data.
- **Network:** The network connects the devices or systems in the distributed system, allowing them to communicate and exchange data.
- **Resource Management:** Distributed systems often have some type of resource management system in place to allocate and manage shared resources such as computing power, storage, and networking.

19. Explain how a Social Media platform can be considered a Distributed Computing System. Identify the key components involved and their roles.

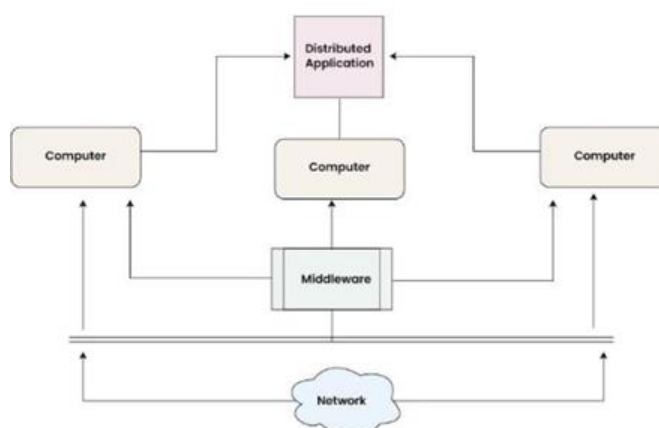
Any Social Media can have its Centralized Computer Network as its Headquarters and computer systems that can be accessed by any user and using their services will be the Autonomous Systems in the Distributed System Architecture.

- **Distributed System Software:** This Software enables computers to coordinate their activities and to share the resources such as Hardware, Software, Data, etc.
- **Database:** It is used to store the processed data that are processed by each Node/System of the Distributed systems that are connected to the Centralized network.

Distributed System



Working of Distributed System



- As we can see that each Autonomous System has a common Application that can have its own data that is shared by the Centralized Database System.
- To Transfer the Data to Autonomous Systems, Centralized System should be having a Middleware Service and should be connected to a Network.
- Middleware Services enable some services which are not present in the local systems or centralized system default by acting as an interface between the Centralized System and the local systems. By using components of Middleware Services systems communicate and manage data.
- The Data which is been transferred through the database will be divided into segments or modules and shared with Autonomous systems for processing.
- The Data will be processed and then will be transferred to the Centralized system through the network and will be stored in the database.

20. Explain the concept of Remote Procedure Calls (RPC) and its working mechanism with its five key elements.

Client: The client process initiates RPC. The client makes a standard call, which triggers a correlated procedure in the client stub.

Client Stub: Stubs are used by RPC to achieve semantic transparency. The client calls the client stub. Client stub does the following tasks:

When the client stub receives a request from a client, it packs(marshalls) the parameters and required specifications of remote/target procedure in a message.

When the client stub receives the result values after execution, it unpacks (unmarshalled) those results and sends them to the Client.

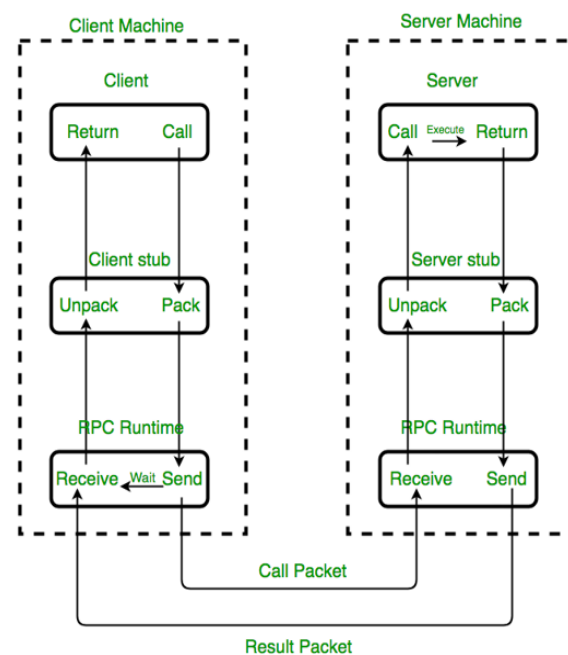
RPC Runtime: The RPC runtime is in charge of message transmission between client and server via the network. Retransmission, acknowledgement, routing, and encryption are all tasks performed by it.

On the client-side, it receives the result values in a message from the server-side, and then it further sends it to the client stub whereas, on the server-side, RPC Runtime got the same message from the server stub when then it forwards to the client machine.

Server Stub: The first task performed by server stub is that it unpacks(unmarshalled) the call request message which is received from the local RPC Runtime and makes a regular call to invoke the required procedure in the server.

The second task performed by server stub is that when it receives the server's procedure execution result, it packs it into a message and asks the local RPC Runtime to transmit it to the client stub where it is unpacked.

Server: After receiving a call request from the client machine, the server stub passes it to the server. The execution of the required procedure is made by the server and finally, it returns the result to the server stub so that it can be passed to the client machine using the local RPC Runtime.



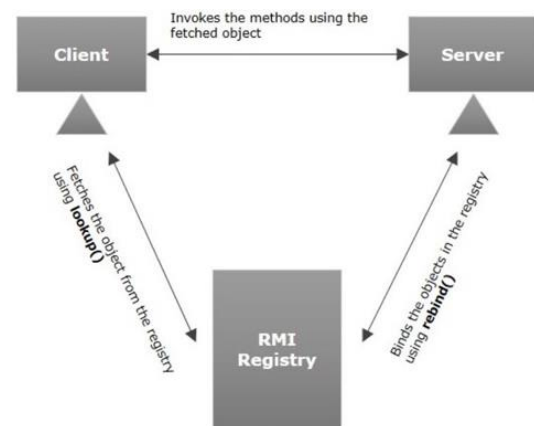
21. What is RMI? Explain its key components.

The Remote Method Invocation (RMI) architecture in Java facilitates communication between Java objects residing in different Java Virtual Machines (JVMs) over a network. It allows objects to invoke methods on remote objects as if they were local objects. The RMI architecture typically involves several components and follows a client-server model:

1. **Remote Interface:** It is a Java interface that defines the methods that can be invoked remotely by clients. It extends the *java.rmi.Remote* interface. Each method in the remote interface must declare *java.rmi.RemoteException* in its throws clause to handle remote method invocation errors.
2. **Remote Object Implementation:** It is a Java class that provides the actual implementation of the methods defined in the remote interface. It is responsible for executing the methods invoked remotely by clients.
3. **RMI Registry:** It is a simple naming service provided by Java RMI that allows clients to look up remote objects by name. It acts as a central registry where remote objects are bound to names, making them accessible to clients. The server creates an RMI registry, and clients use it to locate remote objects.
4. **Client:** It is the application or component that initiates communication with remote objects. It obtains a reference to the remote object from the RMI registry using the naming service provided by *java.rmi.Naming*. Once it has the reference, the client can invoke methods on the remote object as if it were a local object.
5. **Marshalling and Unmarshalling:** RMI handles the marshalling (serialization) and unmarshalling (deserialization) of method parameters and return values transparently to the client and server.
6. **Network Communication:** RMI uses TCP/IP as the underlying network protocol for communication between the client and server JVMs. It establishes a connection between the client and server JVMs, allowing them to exchange method invocations and data.

22. Write short note on RMI Registry.

- RMI registry is a namespace on which all server objects are placed.
- Each time the server creates an object, it registers this object with the RMI registry (using *bind()* or *reBind()* methods).
- These are registered using a unique name known as bind name. It allows remote clients to get a reference to these objects.
- To invoke a remote object, the client needs a reference of that object. At that time, the client fetches the object from the registry using its bind name (using *lookup()* method).



23. Outline the key steps involved in developing a basic Remote Method Invocation (RMI) application.

In this example, the request specifies two numbers. The server adds these together and returns the sum.

Step One: Enter and Compile the Source Code

This application uses four source files.

AddServerIntf.java, defines the remote interface that is provided by the server.

AddServerImpl.java, implements the remote interface.

AddServer.java, contains the main program for the server machine.

AddClient.java, implements the client side of this distributed application.

Compile all the four source code file using javac.

Step Two: **Generate a Stub**

To generate a stub, we use a tool called the RMI compiler, which is invoked from the command line, as shown here: *rmic AddServerImpl*. This command generates the file *AddServerImpl_Stub.class*.

Step Three: **Install Files on the Client and Server Machines**

Copy *AddClient.class*, *AddServerImpl_Stub.class*, and *AddServerIntf.class* to a directory *Stub.class*, and *AddServer.class* to a directory on the server machine.

Step Four: **Start the RMI Registry on the Server Machine**

Start the RMI Registry from the command line, as shown here: *start rmiregistry*

When this command returns, a new window has been created.

Step Five: **Start the Server**

The server code is started from the command line, as shown here: *java AddServer*

Step Six: **Start the Client**

The Add Client software requires three arguments: the name or IP address of the server machine and the two numbers that are to be summed together. We may invoke it from the command line by using one of the two formats shown here: *java AddClient server1 8 9 java*
AddClient 11.12.13.14 8 9

In the first line, the name of the server is provided. The second line uses its IP address (11.12.13.14)

Sample output from this program is shown here:

The first number is: 8
The second number is: 9
The sum is: 17.0

24. Explain the Advantages and Disadvantages distributed computing.

Advantages of the Distributed Computing System are:

- **Scalability:** Distributed systems are generally more scalable than centralized systems, as they can easily add new devices or systems to the network to increase processing and storage capacity.
- **Reliability:** Distributed systems are often more reliable than centralized systems, as they can continue to operate even if one device or system fails.
- **Flexibility:** Distributed systems are generally more flexible than centralized systems, as they can be configured and reconfigured more easily to meet changing computing needs.

There are a few limitations to Distributed Computing System

- **Complexity:** Distributed systems can be more complex than centralized systems, as they involve multiple devices or systems that need to be coordinated and managed.
- **Security:** It can be more challenging to secure a distributed system, as security measures must be implemented on each device or system to ensure the security of the entire system.
- **Performance:** Distributed systems may not offer the same level of performance as centralized systems, as processing and data storage is distributed across multiple devices or systems.